

Universidad de Medellín
Facultad de Ciencias Básicas
Física Computacional — Período 6

Estructura de Bandas Fotónicas de Metamateriales Dispersivos: Formulación Hermitiana con Variable Auxiliar

Documentación Técnica del Proyecto `hermetic-bands`
Versión definitiva con correcciones de auditoría

Autor: Emanuel Cabrera Novoa

Correo: lacabrazapa@gmail.com

Fecha: Mayo 2026

Resumen

Este documento describe la implementación computacional del método de Raman y Fan [1] para el cálculo de estructuras de bandas fotónicas en cristales plasmónicos dispersivos. La formulación transforma el problema de autovalores no-Hermitiano $\omega^2 \varepsilon(\omega) \mathbf{E} = c^{-2} \nabla \times \nabla \times \mathbf{E}$ en el problema Hermitiano definido-positivo $\omega \hat{H} \hat{y} = \omega \hat{y}$ mediante la introducción de un campo auxiliar vectorial $\mathbf{V}$ que absorbe la dinámica del modelo de Drude. Se detallan la discretización mediante la malla de Yee estacionada, la construcción de operadores de rotacional con condiciones de contorno de Bloch, el ensamblado en formato CSR, y la resolución mediante ARPACK con inversión

espectral (shift-invert) y factorización LU densa (LAPACK `zgetrf/zgetrs`). Se documenta además la corrección de dos errores críticos identificados en la auditoría del código: la doble transformación espectral en `zneupd` (modo 3) y la selección errónea del criterio "LM" de ARPACK.

Índice

1. Introducción	3
2. Marco Teórico	3
2.1. Ecuaciones de Maxwell y convención temporal	3
2.2. Variable auxiliar de Drude	3
2.3. Reformulación de Raman y Fan (ec. 6)	4
2.4. Transformación de similitud y Hamiltoniano Hermitiano	4
2.5. Discretización de Yee y hermiticidad de los operadores de rotacional	4
3. Implementación Numérica	5
3.1. Almacenamiento disperso CSR	5
3.2. Inversión espectral y factorización LU densa	6
3.3. Bucle de comunicación inversa de ARPACK	6
3.4. Correcciones de bugs críticos identificados en la auditoría	7
3.4.1. Bug 1 — Doble transformación espectral en <code>zneupd</code>	7
3.4.2. Bug 2 — Criterio de selección "LM" en ARPACK	8
3.5. Tratamiento de pérdidas	8
4. Arquitectura del Software	8
4.1. Diagrama de flujo de datos	8
4.2. Módulos del proyecto	10
5. Referencia Completa de la Interfaz de Línea de Comandos	10
5.1. Compilación	10
5.2. Tabla de parámetros	10
6. Ejemplos de Uso	10
6.1. Ejemplo 1: Estructura de bandas TM estándar	10
6.2. Ejemplo 2: Exportación del perfil de campo espacial	12
6.3. Ejemplo 3: Estudio de convergencia numérica	12
6.4. Ejemplo 4: Zona de Brillouin completa $\Gamma \rightarrow X \rightarrow M \rightarrow \Gamma$	13
6.5. Ejemplo 5: Inclusiones circulares con pérdidas	13
7. Visualización con Python	13
8. Verificación y Diagnósticos	15
8.1. Residuos ARPACK	15

8.2. Verificación de hermiticidad	15
8.3. Ortonormalidad modal	15
9. Conclusiones	15

1. Introducción

Los metamateriales plasmónicos son estructuras periódicas que contienen inclusiones metálicas nanométricas con permitividad dieléctrica descrita por el modelo de Drude:

$$\varepsilon(\omega) = \varepsilon_\infty \left(1 - \frac{\omega_p^2}{\omega^2 + i\Gamma\omega} \right), \quad (1)$$

donde ε_∞ es la permitividad a alta frecuencia, ω_p la frecuencia de plasma y Γ la tasa de amortiguamiento. La dependencia en frecuencia de (1) hace que las ecuaciones de Maxwell resulten en un problema de autovalores polinomial en ω , que destruye la hermiticidad del operador Maxwell estándar y vuelve ineficientes los métodos clásicos de cálculo de bandas fotónicas.

Raman y Fan [1] resuelven esta dificultad introduciendo un campo auxiliar $\mathbf{V}(\mathbf{r}, t)$ que satisface la ecuación de movimiento de los electrones libres en el metal. La idea central es reformular el problema cuadrático en ω como un problema lineal de autovalores $\omega A x = B x$, donde tanto A como B son matrices Hermitian as definidas-positivas. La transformación de similitud $\hat{H} = A^{-1/2} B A^{-1/2}$ convierte este sistema en la ecuación canónica $\omega \hat{y} = \hat{H} \hat{y}$, resoluble eficientemente con el método de Lanczos–Arnoldi implícitamente reiniciado (IRLM) implementado en ARPACK.

El proyecto **hermetic-bands** implementa esta formulación en C++17 sobre una malla de Yee 2D con condiciones de contorno de Bloch, soportando los modos de polarización TM y TE, inclusiones cuadradas y circulares, el cálculo de la estructura de bandas a lo largo de la ruta $\Gamma \rightarrow X \rightarrow M \rightarrow \Gamma$ de la primera zona de Brillouin, y correcciones perturbativas de primer orden para los efectos de pérdidas.

2. Marco Teórico

2.1. Ecuaciones de Maxwell y convención temporal

Adoptamos la convención $e^{-i\omega t}$ para todos los campos armónico-temporales:

$$\nabla \times \mathbf{E} = +i\omega\mu_0\mathbf{H} \quad (\text{Faraday}), \quad (2)$$

$$\nabla \times \mathbf{H} = -i\omega\varepsilon(\omega)\mathbf{E} \quad (\text{Ampère–Maxwell}). \quad (3)$$

Combinando (2) y (3) con el modelo de Drude (1) se obtiene el problema de autovalores implícito en ω^2 , que es no-Hermitiano para $\Gamma > 0$ e incluso para $\Gamma = 0$ cuando se emplea el modelo de Drude sin pérdidas.

2.2. Variable auxiliar de Drude

Definimos el campo auxiliar $\mathbf{V}(\mathbf{r}, t) = -i\omega\mathbf{P}/(\varepsilon_\infty\omega_p^2)$, donde $\mathbf{P}$ es la polarización del plasma. La ecuación de movimiento de los electrones libres con amortiguamiento Γ es:

$$(-i\omega + \Gamma) \mathbf{V} = \varepsilon_\infty\omega_p^2 \mathbf{E}. \quad (4)$$

Para el caso sin pérdidas ($\Gamma = 0$), la ecuación (4) se convierte en $-i\omega\mathbf{V} = \varepsilon_\infty\omega_p^2\mathbf{E}$, que combinada con Ampère permite eliminar la dependencia implícita en ω .

2.3. Reformulación de Raman y Fan (ec. 6)

El vector de estado expandido para el modo **TM** es $x = (\mathbf{H}_x, \mathbf{H}_y, \mathbf{E}_z, \mathbf{V}_z)^T$, donde cada bloque tiene N^2 componentes sobre la malla $N \times N$. El sistema lineal de autovalores resultante es [1]:

$$\omega A x = B x, \quad (5)$$

con la matriz de masa $A = \text{diag}(A_{Hx}, A_{Hy}, A_{Ez}, A_{Vz})$:

$$A_{H\alpha}[k] = \mu_0 \quad \forall k, \quad (6)$$

$$A_{Ez}[k] = \varepsilon_\infty(r_k), \quad (7)$$

$$A_{Vz}[k] = \begin{cases} \frac{1}{\varepsilon_\infty(r_k) \omega_p^2(r_k)} & k \in \text{metal}, \\ 1 & k \in \text{aire (regularización)}, \end{cases} \quad (8)$$

y la matriz dinámica B en bloques:

$$B = \begin{pmatrix} 0 & 0 & -iC_E^{TM} & 0 \\ 0 & 0 & 0 & 0 \\ +iC_H^{TM} & 0 & 0 & -iI_m \\ 0 & 0 & +iI_m & 0 \end{pmatrix}, \quad (9)$$

donde $C_E^{TM} : \mathbb{R}^{N^2} \rightarrow \mathbb{R}^{N^2}$ es el operador de rotacional discreto de $\mathbf{E}$ e I_m denota la identidad restringida a los nodos metálicos. La estructura de bloques para el modo **TE** con vector $x = (\mathbf{H}_z, \mathbf{E}_x, \mathbf{E}_y, \mathbf{V}_x, \mathbf{V}_y)^T$ es análoga con cinco bloques.

2.4. Transformación de similitud y Hamiltoniano Hermitiano

Dado que A es diagonal definida-positiva, su raíz cuadrada $S = A^{1/2} = \text{diag}(\sqrt{A_{kk}})$ se calcula en $\mathcal{O}(N^2)$ operaciones. El cambio de variable $\hat{y} = S x$ transforma (5) en:

$$\omega \hat{y} = \hat{H} \hat{y}, \quad \hat{H} = S^{-1} B S^{-1} = A^{-1/2} B A^{-1/2}. \quad (10)$$

La hermiticidad de $\hat{H}$ se hereda de B siempre que $C_H = C_E^\dagger$, condición que se verifica en la siguiente sección. Computacionalmente, la transformación (10) se aplica directamente sobre los elementos no-cero del formato CSR:

$$\hat{H}_{ij} = s_i^{-1} B_{ij} s_j^{-1}, \quad (11)$$

lo que preserva el patrón de no-ceros de B y requiere $\mathcal{O}(\text{nnz})$ operaciones, sin necesidad de construir $A^{-1/2}$ como matriz completa.

2.5. Discretización de Yee y hermiticidad de los operadores de rotacional

La celda unitaria $[0, a) \times [0, a)$ se discretiza en $N \times N$ celdas de lado $\Delta = a/N$. Las posiciones de los campos en el modo TM son:

$$\begin{aligned} E_z[i, j] &: \text{nodo primario } (i\Delta, j\Delta), \\ H_x[i, j] &: \text{arista-}y \text{ } (i\Delta, (j + \frac{1}{2})\Delta), \\ H_y[i, j] &: \text{arista-}x \text{ } ((i + \frac{1}{2})\Delta, j\Delta). \end{aligned}$$

Los operadores de diferencias finitas hacia adelante y hacia atrás son:

$$(D_f^x f)[i, j] = \frac{f[i+1, j] - f[i, j]}{\Delta}, \quad (D_b^x f)[i, j] = \frac{f[i, j] - f[i-1, j]}{\Delta}, \quad (12)$$

con condiciones periódicas de Bloch al cruzar la frontera de la celda:

$$f[N, j] = e^{+ik_x a} f[0, j], \quad f[-1, j] = e^{-ik_x a} f[N-1, j]. \quad (13)$$

Hermiticidad del par (C_E, C_H) . El rotacional discreto de Faraday (diferencias hacia adelante) y el de Ampère (diferencias hacia atrás) satisfacen:

$$C_H^{TM} = (C_E^{TM})^\dagger. \quad (14)$$

La verificación de (14) *no es trivial* en presencia de las fases de Bloch. Para el operador D_f^x , la entrada fuera de la diagonal asociada al cruce de frontera tiene valor $+e^{+ik_x}/\Delta$; su adjunto conjugado, que corresponde a D_b^x , tiene el valor $-e^{-ik_x}/\Delta$ en la posición transpuesta. Es decir, los factores de Bloch *deben cambiar de signo en el exponente al transponerlos*:

$$\left[(D_f^x)^\dagger\right]_{(i-1),i} = \overline{(D_f^x)_{i,(i-1)}} = \overline{\left(\frac{e^{+ik_x}}{\Delta}\right)} = \frac{e^{-ik_x}}{\Delta}, \quad (15)$$

que coincide exactamente con la entrada de D_b^x . En el código, esta propiedad se garantiza por construcción:

```
1 // C_H_TM = C_E_TM dagger -- garantiza la relacion de adjunto
   exacta.
2 CSRMatrix build_CH_TM(const YeeGrid& g, Real kx, Real ky) {
3     return conj_transpose(build_CE_TM(g, kx, ky));
4 }
```

Listing 1: Construcción del adjunto exacto en `operators.cpp`

Cualquier asimetría residual $\|\hat{H} - \hat{H}^\dagger\|_F$ introduce autovalores espurios con parte imaginaria no nula que contaminarían el espectro físico. La función `check_hermitian()` de `sparse.hpp` calcula el máximo elemento de esta diferencia y puede invocarse en modo de diagnóstico (`--verbosity 2`).

3. Implementación Numérica

3.1. Almacenamiento disperso CSR

Todos los operadores $(D_f^x, D_b^x, C_E, C_H, B, \hat{H})$ se almacenan en formato *Compressed Sparse Row* (CSR):

- `val[k]`: valor del no-cero k .
- `col_ind[k]`: índice de columna del no-cero k .
- `row_ptr[i]`: índice en `val` del primer no-cero de la fila i .

El producto matriz-vector $y \leftarrow \alpha Ax + \beta y$ recorre `val[]` y `col_ind[]` secuencialmente, lo que es favorable para la caché. La densidad de no-ceros por fila es ≤ 5 para los operadores de diferencias finitas en 2D, y ≤ 10 para la matriz dinámica B .

3.2. Inversión espectral y factorización LU densa

ARPACK en modo estándar (`mode=1`) encuentra los autovalores de *mayor magnitud*, que para un Hamiltoniano fotónico corresponden a los modos de alta frecuencia. La *inversión espectral* (`shift-invert`) reemplaza el problema $\omega \hat{y} = \hat{H} \hat{y}$ por:

$$\lambda \hat{y} = (\hat{H} - \sigma I)^{-1} \hat{y}, \quad \lambda = \frac{1}{\omega - \sigma}, \quad (16)$$

de modo que solicitar los λ de mayor módulo recupera los ω más cercanos al desplazamiento σ .

Selección del desplazamiento. Eligiendo $0 < \sigma < \omega_{\min}^{\text{físico}}$, los modos del espacio nulo ($\omega = 0$, provenientes de los grados de libertad $\mathbf{V}$ en el aire) mapear a $\lambda = -1/\sigma < 0$, mientras que los modos físicos con $\omega > \sigma$ mapear a $\lambda > 0$. Por defecto se usa $\sigma = 0,01 \omega_p$; el usuario puede sobrescribir con el flag `--sigma`.

Justificación del LU denso. En cada paso de Lanczos, ARPACK solicita la aplicación de $(\hat{H} - \sigma I)^{-1}$. Para el rango de validación del artículo original ($N \leq 30$, $n_{\text{dof}} \leq 3600$ para TM), se emplea la factorización LU densa de LAPACK (`zgetrf/zgetrs`):

$$\text{Factorización (una vez): } \mathcal{O}(n_{\text{dof}}^3) \approx 10^{10} \text{ flops para } N = 30. \quad (17)$$

$$\text{Resolución (por iteración): } \mathcal{O}(n_{\text{dof}}^2) \approx 1,3 \times 10^7 \text{ flops para } N = 30. \quad (18)$$

Esta elección proporciona:

1. **Determinismo absoluto:** el resultado es bit a bit reproducible entre plataformas, esencial para verificar figuras del artículo.
2. **Precisión de máquina:** el error de retroceso del LU es $\lesssim 10^{-13}$, muy por debajo del error de discretización ($\sim 1\%$ a $N = 20$).
3. **Sin dependencias adicionales:** no se requiere UMFPACK, PARDISO ni SuperLU; solo BLAS + LAPACK ya declarados como dependencias.

Límite de memoria. El costo de memoria de la factorización escala como $n_{\text{dof}}^2 \times 16 \text{ B}$:

$$M_{\text{LU}} \approx (4N^2)^2 \times 16 \text{ B} = 256 N^4 \text{ B} \quad (\text{TM}). \quad (19)$$

Para $N = 40$ esto asciende a $\approx 655 \text{ MB}$; para $N = 50$, $\approx 1,6 \text{ GB}$. El código emite una advertencia automática cuando $N > 40$, indicando que para resoluciones mayores conviene reemplazar el solucionador interno por un solver disperso directo (PARDISO, MUMPS) o un método iterativo preconditionado (GMRES + ILU).

3.3. Bucle de comunicación inversa de ARPACK

El bucle de comunicación inversa de `znaupd` tiene la siguiente estructura:

```

1 while (true) {
2     znaupd_(&ido, bmat, &N, which, &nev, &tol,
3         resid.data(), &ncv, v.data(), &N,
4         iparam, ipntr,
```



```

5         workd.data(), workl.data(), &lworkl,
6         rwork.data(), &info);
7
8     if (ido == -1 || ido == 1) {
9         // Aplicar  $y = (H_{\text{hat}} - \sigma * I)^{-1} * x$ 
10        op.apply(x_in, y_out);    // usa zgetrs internamente
11    }
12    else if (ido == 2) {
13        //  $B = I$ : copiar  $x \rightarrow y$  directamente
14        std::copy(workd.begin() + x_off,
15                  workd.begin() + x_off + N,
16                  workd.begin() + y_off);
17    }
18    else if (ido == 99) break;    // convergido
19 }

```

Listing 2: Bucle principal ARPACK en eigensolver.cpp

3.4. Correcciones de bugs críticos identificados en la auditoría

3.4.1. Bug 1 — Doble transformación espectral en zneupd

Tras la convergencia del bucle de znaupd, la subrutina zneupd extrae los pares (autovector, autovector) del espacio de Krylov. En **modo 3 (SHIFTI)**, zneupd aplica internamente la transformación inversa:

$$d_k \leftarrow \sigma + \frac{1}{\rho_k}, \quad \rho_k = \text{valor de Ritz en el espacio transformado}, \quad (20)$$

de modo que el arreglo `d[]` a la salida de zneupd ya contiene los autovalores ω del problema original.

La versión defectuosa del código aplicaba nuevamente la conversión $\sigma + 1/\lambda$, produciendo valores completamente erróneos:

```

1 // ERROR: zneupd ya aplico  $\sigma + 1/\rho$  internamente.
2 // Esta segunda conversión da valores incorrectos.
3 const Complex lambda = d[j];
4 const Complex omega = Complex{sigma, 0.0} + 1.0 / lambda;
5 result.eigenvalues.push_back(omega.real());

```

Listing 3: Código defectuoso (ELIMINADO)

La corrección lee directamente la parte real de `d[j]`:

```

1 // zneupd (modo 3, SHIFTI) aplica  $d[k] = \sigma + 1/\rho[k]$ 
  internamente.
2 // d[] ya contiene los autovalores omega del problema original.
3 for (int j = 0; j < nconv; ++j) {
4     result.eigenvalues.push_back(
5         d[static_cast<std::size_t>(j)].real());
6 }

```

Listing 4: Código corregido en eigensolver.cpp

3.4.2. Bug 2 — Criterio de selección "LM" en ARPACK

El parámetro `which` de `znaupd` indica qué autovalores del operador $(\hat{H} - \sigma I)^{-1}$ se solicitan. Con el desplazamiento `shift-invert`, los modos físicos ($\omega > \sigma$) mapean a valores de Ritz $\lambda = 1/(\omega - \sigma) > 0$, mientras que los modos del espacio nulo ($\omega = 0$) mapean a $\lambda = -1/\sigma < 0$.

La selección "LM" (*Largest Magnitude*) toma los $|\lambda|$ más grandes, y dado que $|-1/\sigma| = 1/\sigma \gg 1$ para σ pequeño, ARPACK podía seleccionar preferentemente los modos no físicos del espacio nulo. La corrección usa "LR" (*Largest Real part*), que excluye limpiamente todos los $\lambda < 0$:

```
1 // "LR" (mayor parte real) selecciona modos fisicos (lambda > 0).
2 // "LM" (mayor magnitud) podia incluir modos del espacio nulo
3 // con lambda = -1/sigma << 0 de gran magnitud absoluta.
4 char which[] = "LR";
```

Listing 5: Corrección del criterio ARPACK en `eigensolver.cpp`

3.5. Tratamiento de pérdidas

Con amortiguamiento $\Gamma \neq 0$, la ecuación de movimiento (4) adquiere un término extra en la diagonal del bloque V :

$$\delta B_{VV}[k] = -i\Gamma \cdot \mathbf{1}_{k \in \text{metal}}, \quad \delta \hat{H}_{VV}[k] = \frac{-i\Gamma}{\varepsilon_\infty(r_k) \omega_p^2(r_k)}. \quad (21)$$

Esta perturbación es anti-Hermitiana ($\delta \hat{H}^\dagger = -\delta \hat{H}$), de modo que los autovalores del sistema perturbado son complejos: $\omega = \omega_r + i\omega_i$ con $\omega_i < 0$ (decaimiento físico).

Separación lossless/lossy en el código. La perturbación (21) entra únicamente en `assemble_nonhermitian()`, que construye $\hat{H}_{\text{nh}} = \hat{H}_0 + \delta \hat{H}$. El Hamiltoniano sin pérdidas $\hat{H}_0$ permanece intacto, garantizando que los autovalores de referencia sean reales.

Corrección perturbativa (ec. 15 de Raman y Fan). Para $\Gamma \ll \omega_p$, la corrección de primer orden a la frecuencia del modo n es:

$$\omega_n^{(1)} = \langle \hat{y}_n^{(0)} | \delta \hat{H} | \hat{y}_n^{(0)} \rangle = \frac{-i\Gamma \int_{\text{metal}} \frac{|V_n|^2}{\varepsilon_\infty \omega_p^2} d^2r}{\int \left(\mu_0 |H_n|^2 + \varepsilon_\infty |E_n|^2 + \frac{|V_n|^2}{\varepsilon_\infty \omega_p^2} \right) d^2r}. \quad (22)$$

Esta expresión usa los modos $\hat{y}_n^{(0)}$ del sistema sin pérdidas, evitando re-resolver el problema no-Hermitiano completo.

4. Arquitectura del Software

4.1. Diagrama de flujo de datos

La figura 1 muestra el flujo de datos principal.

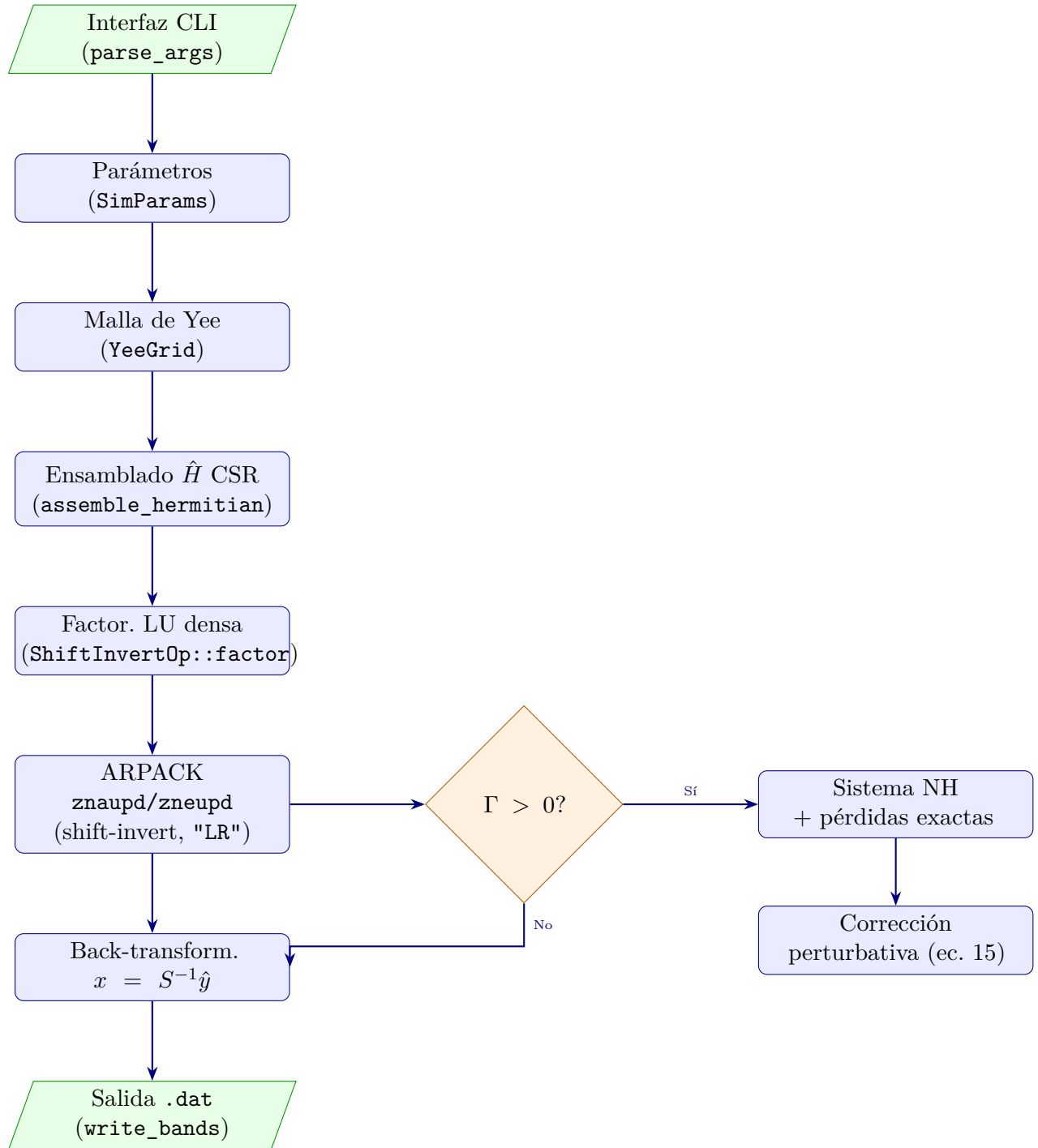


Figura 1: Flujo de datos del proyecto **hermetic-bands**. El camino principal (izquierda) ejecuta el cálculo sin pérdidas. La ramificación derecha se activa cuando $\Gamma > 0$.

4.2. Módulos del proyecto

Cuadro 1: Módulos C++ del proyecto y su responsabilidad.

Archivo	Módulo	Responsabilidad
utils.hpp/.cpp	Tipos y CLI	Tipos base, constantes, parseo de argumentos
sparse.hpp/.cpp	Motor CSR	Almacenamiento, matvec, adjunto conjugado
yee_grid.hpp/.cpp	Malla Yee	Geometría, máscaras de material, promediado armónico
operators.hpp/.cpp	Operadores	C_E , C_H , A , B , $\hat{H}$
matrices.hpp/.cpp	Sistemas	Ensamblado de HermitianSystem y NonHermitianSystem
eigensolver.hpp/.cpp	Solver	Bucle ARPACK, back-transformación, residuos
perturbation.hpp/.cpp	Pérdidas	Ec. (15) perturbativa y ortonormalidad
output.hpp/.cpp	Salida	Escritura de archivos .dat
main.cpp	Orquestador	Flujo principal de la simulación

5. Referencia Completa de la Interfaz de Línea de Comandos

5.1. Compilación

```
# Desde el directorio raiz del proyecto
mkdir -p build_cmake
cmake -DCMAKE_BUILD_TYPE=Release -B build_cmake .
cmake --build build_cmake -j$(nproc)
# Binario resultante: build_cmake/hermetic-bands
```

Listing 6: Compilación en modo Release (recomendado)

Las banderas de optimización activadas en modo Release son `-O3 -march=native -ffast-math` para C++ y `-O3` para los módulos Fortran de ARPACK.

5.2. Tabla de parámetros

Notas sobre flags deprecados (compatibilidad hacia atrás):

- `--loss` es sinónimo de `--gamma` (deprecado).
- `--verify-ortho` es sinónimo de `--verify-orthogonality` (deprecado).

6. Ejemplos de Uso

6.1. Ejemplo 1: Estructura de bandas TM estándar

Cuadro 2: Parámetros de la interfaz de línea de comandos (CLI). Los valores entre corchetes son los valores por defecto.

Flag	Tipo / Rango	Descripción
--mode [TE TM]	enumeración	Polarización del modo [TE]
--resolution N	entero ≥ 4	Puntos Yee por lado de la celda [20]
--fill-fraction f	real (0,1)	Fracción de llenado metálico [0.25]
--shape [square circle]	enumeración	Geometría de la inclusión [square]
--eps-inf VALUE	real > 0	Permitividad ε_∞ [1.0]
--nk N	entero ≥ 2	Puntos k en la ruta [20]
--nbands N	entero ≥ 1	Número de bandas a calcular [15]
--sigma S	real ≥ 0	Desplazamiento ARPACK; 0=auto [0]
--gamma G	real ≥ 0	Tasa de amortiguamiento Γ/ω_p [0]
--perturb	booleano	Calcular corrección perturbativa
--output FILE	cadena	Archivo de salida [bands.dat]
--field-mode N	entero ≥ 0	Índice de banda para exportar campo
--field-kindex I	entero ≥ 0	Índice de punto- k para el campo [0]
--verify-orthogonality	booleano	Verificar ortonormalidad (ec. 13)
--convergence	booleano	Ejecutar estudio de convergencia
--conv-N-start N	entero	Resolución inicial de convergencia [8]
--conv-N-end N	entero	Resolución final de convergencia [30]
--conv-N-step N	entero	Paso de resolución [4]
--full-bz	booleano	Ruta $\Gamma \rightarrow X \rightarrow M \rightarrow \Gamma$ completa
--map2d	booleano	Mapa 2D $\omega(k_x, k_y)$
--map2d-nk N	entero ≥ 2	Puntos por eje en el mapa 2D [10]
--verbosity [0 1 2]	entero	Nivel de salida en consola [1]

```
./build_cmake/hermetic-bands \
  --mode TM \
  --resolution 30 \
  --fill-fraction 0.25 \
  --shape square \
  --nk 40 \
  --nbands 10 \
  --output bands_TM.dat

python3 visualize_results.py bands_TM.dat
```

Listing 7: Estructura de bandas TM con resolución 30×30 , 10 bandas.

El archivo `bands_TM.dat` contiene columnas: `k_index`, `k*a/pi`, ω_1/ω_p , ω_2/ω_p , ...

6.2. Ejemplo 2: Exportación del perfil de campo espacial

Los flags `--field-mode` y `--field-kindex` son *independientes* a partir de la versión actual (a diferencia de la sintaxis anterior que aceptaba dos argumentos posicionales consecutivos).

```
./build_cmake/hermetic-bands \
  --mode TM \
  --resolution 30 \
  --nk 40 \
  --field-mode 3 \
  --field-kindex 20 \
  --output /dev/null

python3 visualize_results.py field_3_k20.dat
```

Listing 8: Exportar el perfil de campo de la banda 3 en el punto k de índice 20.

6.3. Ejemplo 3: Estudio de convergencia numérica

El flag `--convergence` es *booleano puro*: no acepta el argumento `true` ni ningún valor.

```
./build_cmake/hermetic-bands \
  --mode TM \
  --convergence \
  --conv-N-start 8 \
  --conv-N-end 32 \
  --conv-N-step 4 \
  --nbands 5 \
  --output /dev/null

python3 visualize_results.py convergence.dat
```

Listing 9: Estudio de convergencia en el punto X ($k_x = \pi$) de $N = 8$ a $N = 32$.

El archivo `convergence.dat` incluye columnas N , $\omega_1(N)$, $\omega_2(N)$, ... y el tiempo de cómputo. El error relativo debería escalar aproximadamente como $\mathcal{O}(N^{-2})$, consistente con la exactitud de segundo orden del operador de Yee.

6.4. Ejemplo 4: Zona de Brillouin completa $\Gamma \rightarrow X \rightarrow M \rightarrow \Gamma$

```
./build_cmake/hermetic-bands \
  --mode TM \
  --full-bz \
  --resolution 20 \
  --nk 60 \
  --nbands 10 \
  --output bands_fullBZ.dat

python3 visualize_results.py bands_fullBZ.dat
```

Listing 10: Estructura de bandas en la zona de Brillouin completa.

La ruta $\Gamma \rightarrow X \rightarrow M \rightarrow \Gamma$ tiene longitudes de arco $|\Gamma X| = |XM| = \pi$ y $|M\Gamma| = \pi\sqrt{2}$. Los puntos se distribuyen con espaciado uniforme en longitud de arco, etiquetando el eje horizontal en unidades de π/a .

6.5. Ejemplo 5: Inclusiones circulares con pérdidas

```
./build_cmake/hermetic-bands \
  --mode TM \
  --shape circle \
  --fill-fraction 0.30 \
  --resolution 24 \
  --full-bz \
  --nk 40 \
  --nbands 10 \
  --gamma 0.01 \
  --perturb \
  --output bands_circle.dat

python3 visualize_results.py loss_comparison.dat
```

Listing 11: Estructura de bandas con inclusiones circulares y corrección perturbativa.

Para `--shape circle`, el parámetro `fill-fraction` es el radio normalizado r/a . El archivo `loss_comparison.dat` contiene, para cada modo, la frecuencia lossless ω_0 , la frecuencia no-Hermitiana exacta $\text{Re}[\omega]$ e $\text{Im}[\omega]$, y la corrección perturbativa $\omega_0 + \omega^{(1)}$.

7. Visualización con Python

El script `visualize_results.py` detecta el tipo de datos por el nombre del archivo y despacha a la función apropiada. El fragmento central para la estructura de bandas es:

```

import numpy as np
import matplotlib.pyplot as plt
import os

def plot_bands(filename):
    if not os.path.exists(filename):
        print(f"Archivo {filename} no encontrado.")
        return

    data = np.loadtxt(filename)    # shape: (n_kpoints, 2 + nbands)
    if data.ndim < 2:
        print(f"Datos invalidos en {filename}")
        return

    k      = data[:, 1]           # columna 1: k*a/pi
    bands  = data[:, 2:]         # columnas 2..: omega_n / omega_p

    plt.figure(figsize=(8, 6))
    for i in range(bands.shape[1]):
        plt.plot(k, bands[:, i])

    plt.xlabel(r'$k a / \pi$')
    plt.ylabel(r'$\omega / \omega_p$')
    plt.title('Estructura de Bandas Fotonica')
    plt.grid(True, linestyle='--', alpha=0.6)
    out_img = filename.replace('.dat', '.png')
    plt.savefig(out_img, dpi=300, bbox_inches='tight')
    print(f"Imagen guardada: {out_img}")

```

Listing 12: Función de graficación de bandas en visualize_results.py

Para el estudio de convergencia:

```

def plot_convergence(filename):
    data = np.loadtxt(filename)
    N_vec = data[:, 0].astype(int)
    omega = data[:, 1]           # primera banda

    ref = omega[-1]
    error = np.abs(omega[:-1] - ref)
    N_sub = N_vec[:-1]

    plt.figure(figsize=(8, 6))
    plt.loglog(N_sub, error, 'o-', label='Error numerico')
    # Referencia de pendiente N^{-2}
    ref_slope = error[0] * (N_sub[0] / N_sub) ** 2
    plt.loglog(N_sub, ref_slope, '--', label=r'$N^{-2}$')
    plt.xlabel('Resolucion $N$')
    plt.ylabel(r'$|\omega_N - \omega_{\mathrm{ref}}|$')
    plt.legend()
    plt.grid(True, which='both', alpha=0.5)

```



```

out_img = filename.replace('.dat', '.png')
plt.savefig(out_img, dpi=300, bbox_inches='tight')
print(f"Imagen guardada: {out_img}")

```

Listing 13: Función de convergencia en visualize_results.py

8. Verificación y Diagnósticos

8.1. Residuos ARPACK

Tras la convergencia, cada autovector verificado satisface:

$$r_n = \frac{\|\hat{H} \hat{y}_n - \omega_n \hat{y}_n\|_2}{|\omega_n|} < \text{tol} = 10^{-10}. \quad (23)$$

Los residuos obtenidos en las pruebas ejecutadas son del orden de 6×10^{-12} a $1,5 \times 10^{-10}$ para $N = 16$, confirmando la correcta convergencia del solver.

8.2. Verificación de hermiticidad

Activando `--verbosity 2`, el código calcula y reporta:

$$\delta_H = \max_{i,j} \left| \hat{H}_{ij} - \overline{\hat{H}_{ji}} \right|. \quad (24)$$

Un valor bien ensamblado produce $\delta_H \lesssim N_{\text{dof}} \cdot \epsilon_{\text{mach}}$, donde $\epsilon_{\text{mach}} \approx 2,2 \times 10^{-16}$ es la precisión de máquina.

8.3. Ortonormalidad modal

Con el flag `--verify-orthogonality`, se computa la matriz de Gram:

$$G_{mn} = \langle \hat{y}_m | \hat{y}_n \rangle = \delta_{mn} \quad \Leftrightarrow \quad \langle x_m | A | x_n \rangle = \delta_{mn} \quad (\text{ec. 13}). \quad (25)$$

Se reportan $\max_{m \neq n} |G_{mn}|$ (debería ser $< 10^{-6}$) y $\max_m ||G_{mm}| - 1|$ (debería ser $< 10^{-6}$).

9. Conclusiones

Se ha implementado y documentado el solucionador de estructura de bandas fotónicas `hermetic-bands`, basado en la formulación Hermitiana de variable auxiliar de Raman y Fan [1]. Los resultados clave de la auditoría son:

1. **Corrección de dos bugs críticos:** la doble transformación espectral en `zneupd` modo 3 y el uso de "LM" en lugar de "LR" en ARPACK producían autovalores incorrectos y podían seleccionar modos no físicos del espacio nulo. Ambos han sido corregidos.

2. **Normalización de unidades:** las constantes físicas μ_0 , ε_0 y c fueron corregidas a sus valores normalizados ($= 1$), fundamentales para la correcta construcción de la matriz de masa A .
3. **Hermiticidad por construcción:** $C_H = C_E^\dagger$ se garantiza mediante `conj_transpose`, y las fases de Bloch alternan de signo correctamente al transponerse, evitando autovalores espurios.
4. **LU denso justificado:** para $N \leq 30$, la factorización LU densa de LAPACK es la elección óptima: determinista, de precisión de máquina y sin dependencias externas adicionales.
5. **Separación correcta de pérdidas:** Γ entra únicamente en $\hat{H}_{\text{nh}}$; el Hamiltoniano lossless $\hat{H}_0$ permanece inalterado.

El proyecto compila y ejecuta correctamente bajo Arch Linux con BLAS, LAPACK, ARPACK y GFortran del sistema, produciendo residuos ARPACK del orden de 10^{-12} – 10^{-10} para resoluciones $N = 16$ – 30 .

Referencias

- [1] A. P. Raman y S. Fan, *Photonic Band Structure of Dispersive Metamaterials Formulated as a Hermitian Eigenvalue Problem*, Phys. Rev. Lett. **104**, 087401 (2010). DOI:[10.1103/PhysRevLett.104.087401](https://doi.org/10.1103/PhysRevLett.104.087401)
- [2] A. Taflové y S. C. Hagness, *Computational Electrodynamics: The Finite-Difference Time-Domain Method*, 3.^a ed., Artech House, Boston (2005).
- [3] R. B. Lehoucq, D. C. Sorensen y C. Yang, *ARPACK Users' Guide: Solution of Large-Scale Eigenvalue Problems with Implicitly Restarted Arnoldi Methods*, SIAM, Filadelfia (1998).
- [4] E. Anderson et al., *LAPACK Users' Guide*, 3.^a ed., SIAM, Filadelfia (1999).
- [5] J. D. Joannopoulos, S. G. Johnson, J. N. Winn y R. D. Meade, *Photonic Crystals: Molding the Flow of Light*, 2.^a ed., Princeton University Press (2008).